Multi-Region Disaster Recovery Plan

Services: EC2, RDS, Route 53, CloudFormation.

Description: Set up a disaster recovery architecture where data and applications are replicated across two regions. Automate failover using Route 53 health checks.

Skills: High availability, failover planning, cross-region replication.

Definition: The Multi-Region Disaster Recovery Plan project demonstrates how to design and implement a highly available and resilient architecture on AWS by deploying applications and databases across two geographically separated regions. Using EC2 for the web application, RDS for the database, and Route 53 for DNS health checks and automated failover, the project ensures business continuity in case of regional failures. This project highlights skills in high availability, failover planning, and cross-region replication.

Scope of the project:

- Deploy EC2 and RDS across two regions for disaster recovery.
- Use Route 53 health checks to enable automatic failover of web traffic.
- Demonstrate database failover using RDS snapshots.
- Ensure high availability and business continuity during regional failures.
- Implement Free Tier-compatible solution for testing and learning.

Methodology:

1.**Manual AWS Management Console Method**

- Uses EC2, RDS, Route 53, Security Groups, and VPC.
- Deploys primary and secondary EC2 instances in different regions.
- Configures Route 53 health checks and failover DNS for automatic traffic redirection.
- Creates RDS snapshots and restores them in the secondary region to simulate database failover.
- Cost-effective for Free Tier or academic projects, allows hands-on understanding of failover and DR concepts.

2.AWS CloudFormation - Automated Method

- Uses CloudFormation templates to automate EC2, RDS, and Route 53 setup.
- Ensures consistent and repeatable deployment across multiple regions.
- Reduces manual configuration errors and speeds up setup.
- Suitable for production environments with larger scale DR requirements.
- Less cost-effective for small or one-time Free Tier projects.

AWS Services Used in the Project:

1.EC2 (Elastic Compute Cloud)

- **Definition:** A scalable virtual server service in the cloud.
- **Role:** Hosts web applications in different regions for disaster recovery.
- **Purpose in Project:** Ensures application availability by providing primary and secondary web servers that handle traffic during failover.

2.**RDS (Relational Database Service – MySQL)**

- **Definition:** Managed database service that simplifies setup, operation, and scaling of relational databases.
- **Role:** Stores application data and supports backup/restore for disaster recovery.

- **Purpose in Project:** Maintains data availability by enabling snapshot-based restoration to a secondary region.

3.**Route 53**

- **Definition:** AWS DNS service that routes user requests to infrastructure endpoints.
- **Role:** Monitors endpoint health and performs failover routing.
- **Purpose in Project:** Automatically redirects web traffic from primary to secondary EC2 instance if the primary fails.

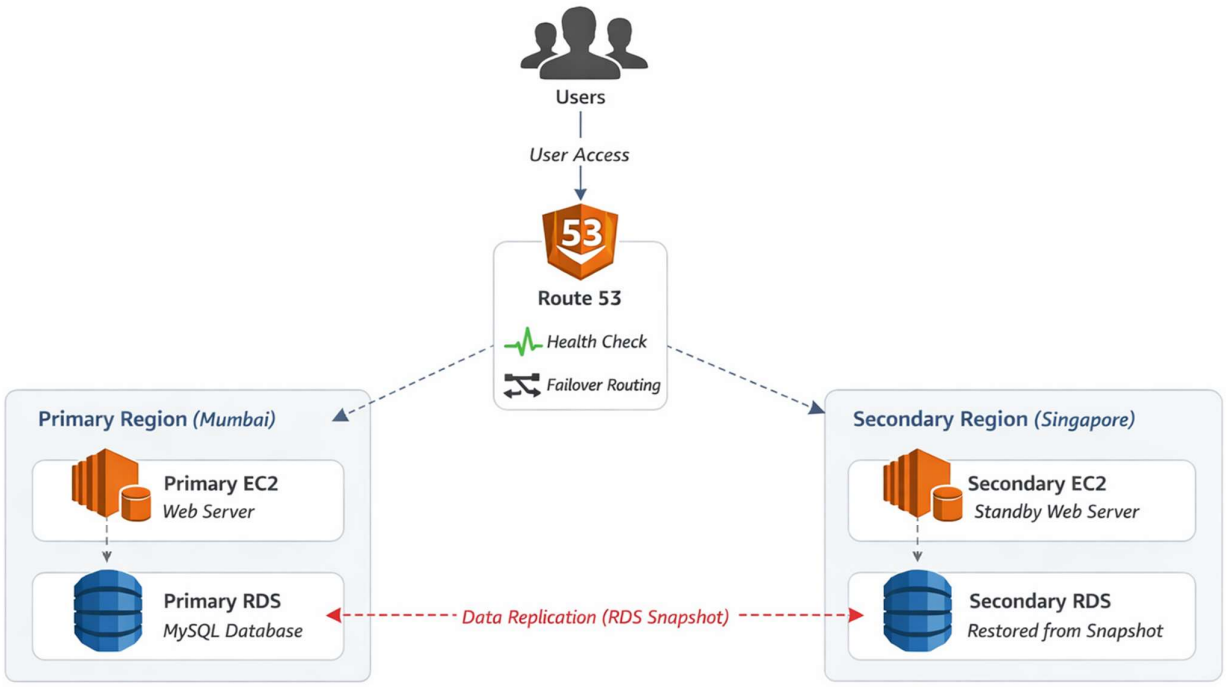
4.**VPC (Virtual Private Cloud) & Security Groups**

- **Definition:** Provides isolated cloud network and firewall rules for resources.
- **Role:** Controls secure network communication and access to resources.
- **Purpose in Project:** Ensures secure connectivity between EC2 and RDS instances while restricting unauthorized access.

5.Snapshots

- **Definition:** Point-in-time backup of RDS databases or EC2 volumes.
- **Role:** Allows replication and recovery of data in another region.
- **Purpose in Project:** Provides disaster recovery by restoring RDS database in the secondary region during failover.

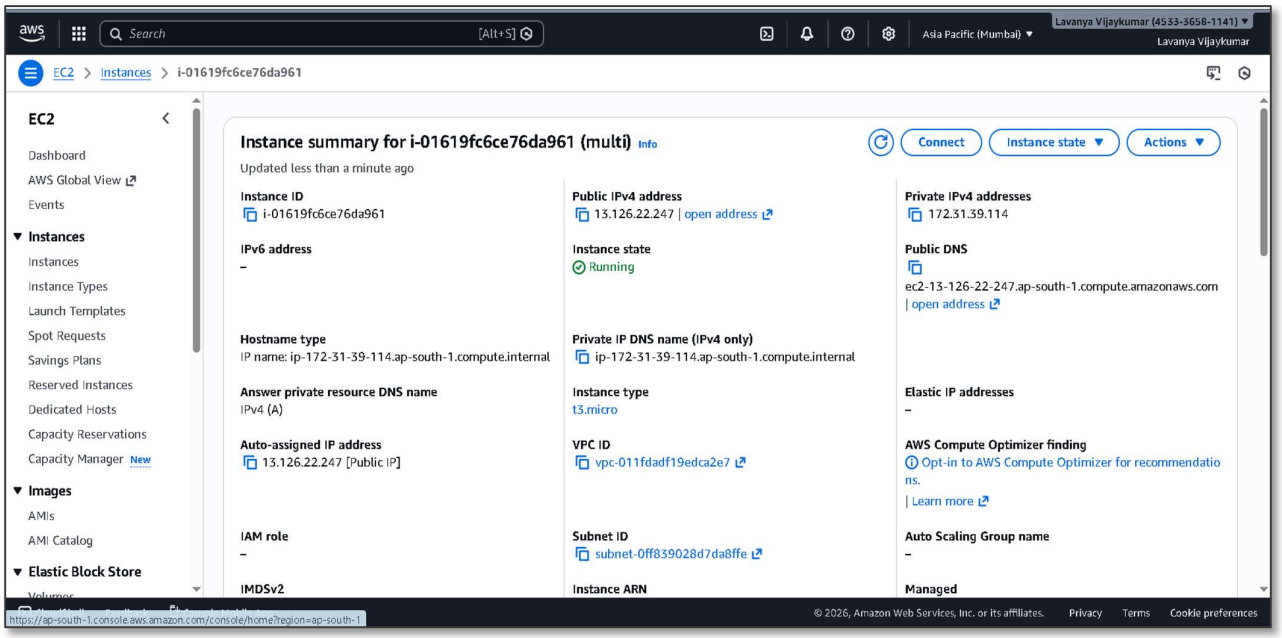
Architecture Diagram:



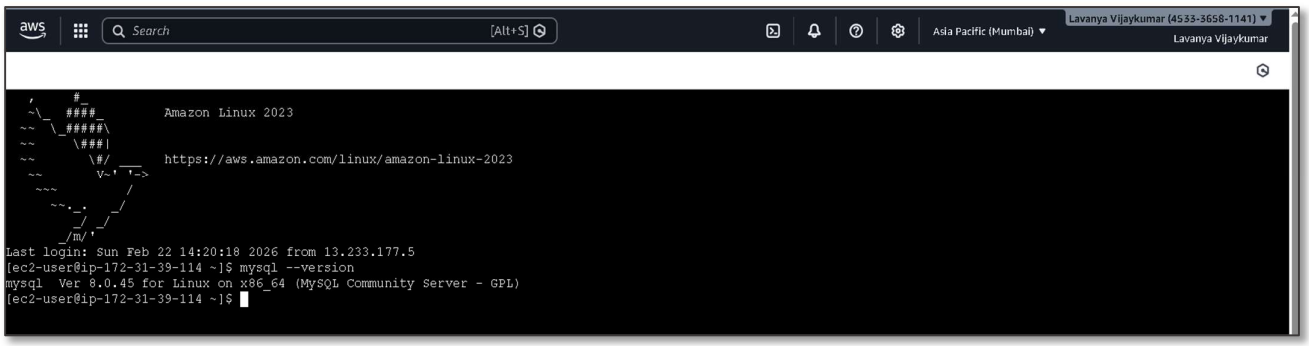
Project Implementation Steps:

STEP 1: Primary EC2 Setup (Mumbai)

- Launch EC2 Instance
- Region: Mumbai (ap-south-1)
- Type: t2.micro (Free Tier)
- OS: Amazon Linux 2023
- Security Group: allow SSH (22), HTTP (80), optional MySQL (3306)

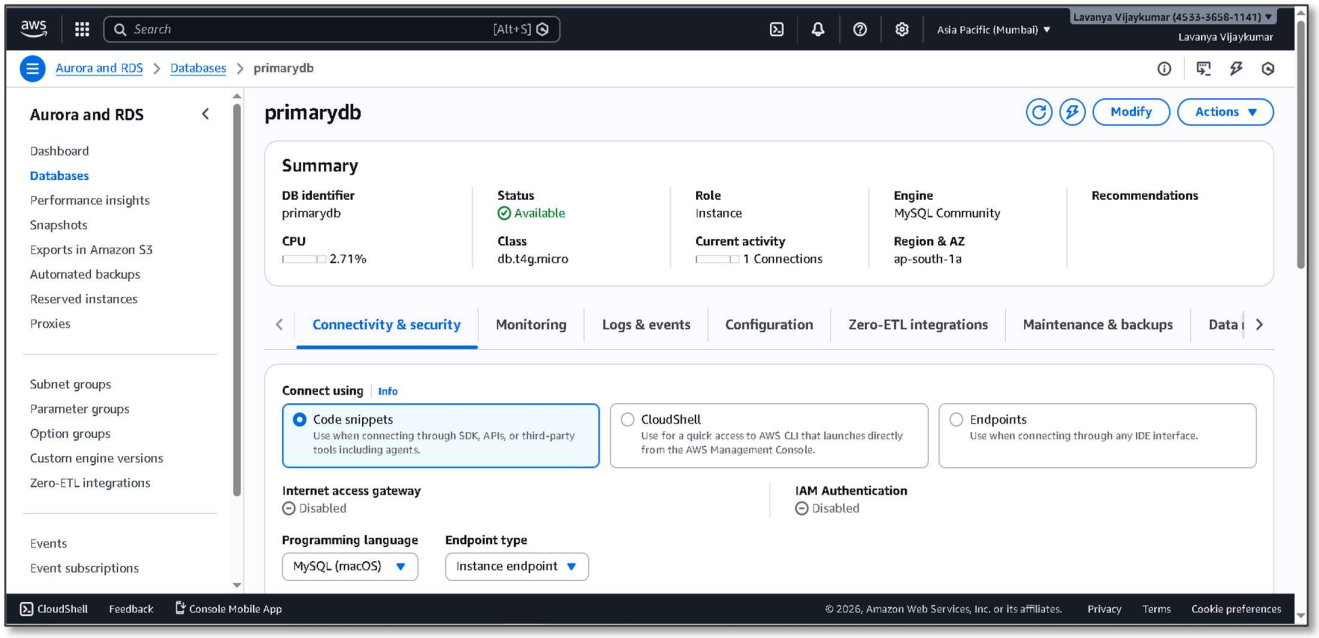


- Connect via SSH and install web server + MySQL client:
- sudo dnf update -y
- sudo dnf install -y httpd
- sudo systemctl enable httpd
- sudo systemctl start httpd
- echo "Primary Region EC2" | sudo tee /var/www/html/index.html
- # Install MySQL client
- sudo dnf install --nogpgcheck -y mysql-community-client

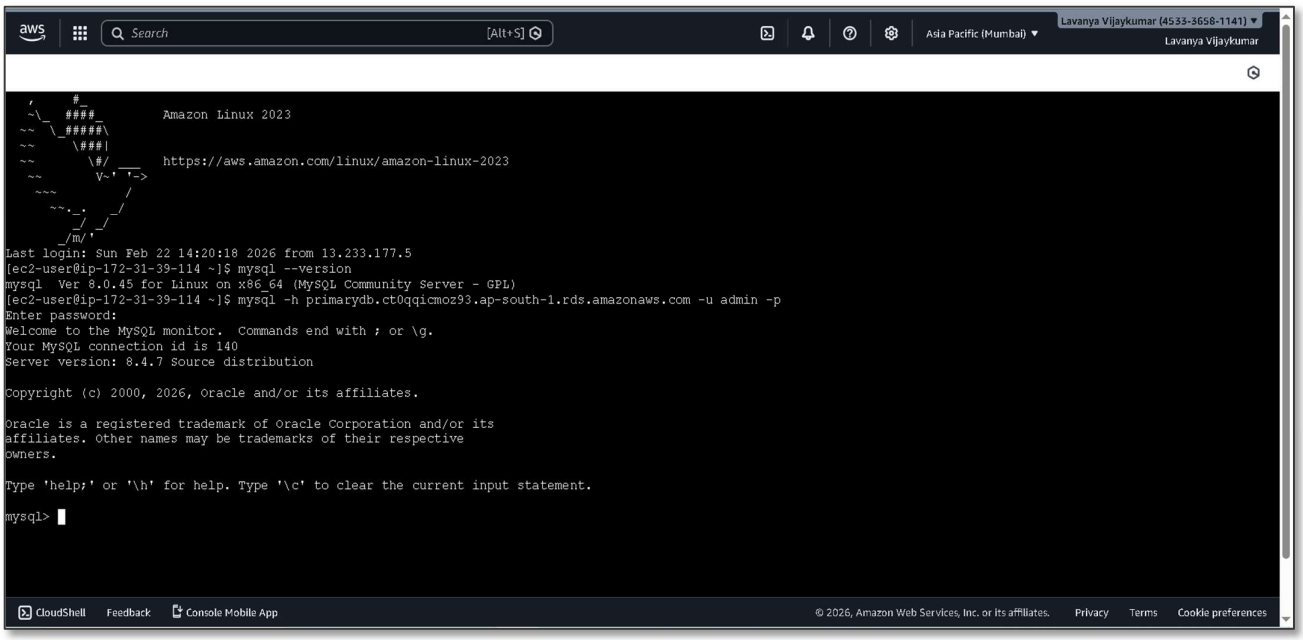


STEP 2:Primary RDS Setup (Mumbai)

- Go to RDS Console → Create Database → MySQL (Free Tier)
- DB Identifier: PrimaryDB
- Master Username: admin
- Publicly Accessible: Yes
- Security Group: allow Primary EC2 IP on port 3306

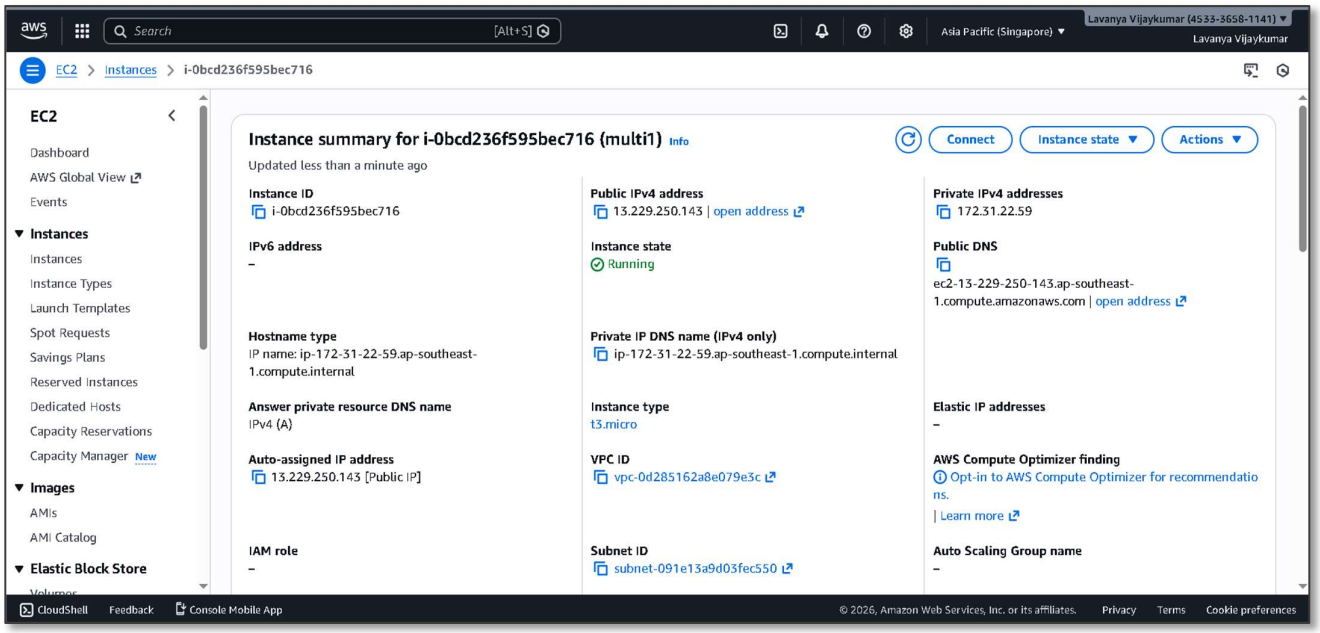


- Connect from Primary EC2:
- mysql -h <PrimaryDB_Endpoint> -u admin -p

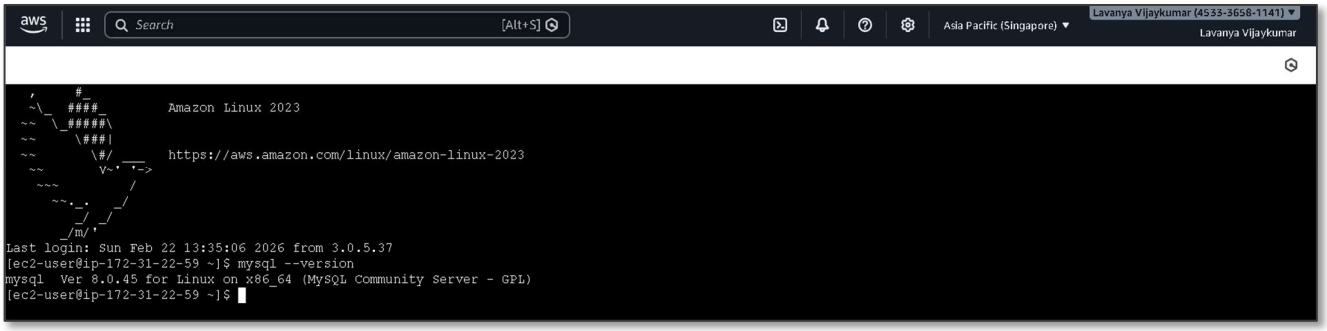


STEP 3:Secondary EC2 Setup (Singapore)

- Launch EC2 in Singapore (ap-southeast-1)
- Type: t2.micro (Free Tier)
- Security Group: allow SSH, HTTP, optional MySQL
- Keep stopped until failover to save costs.

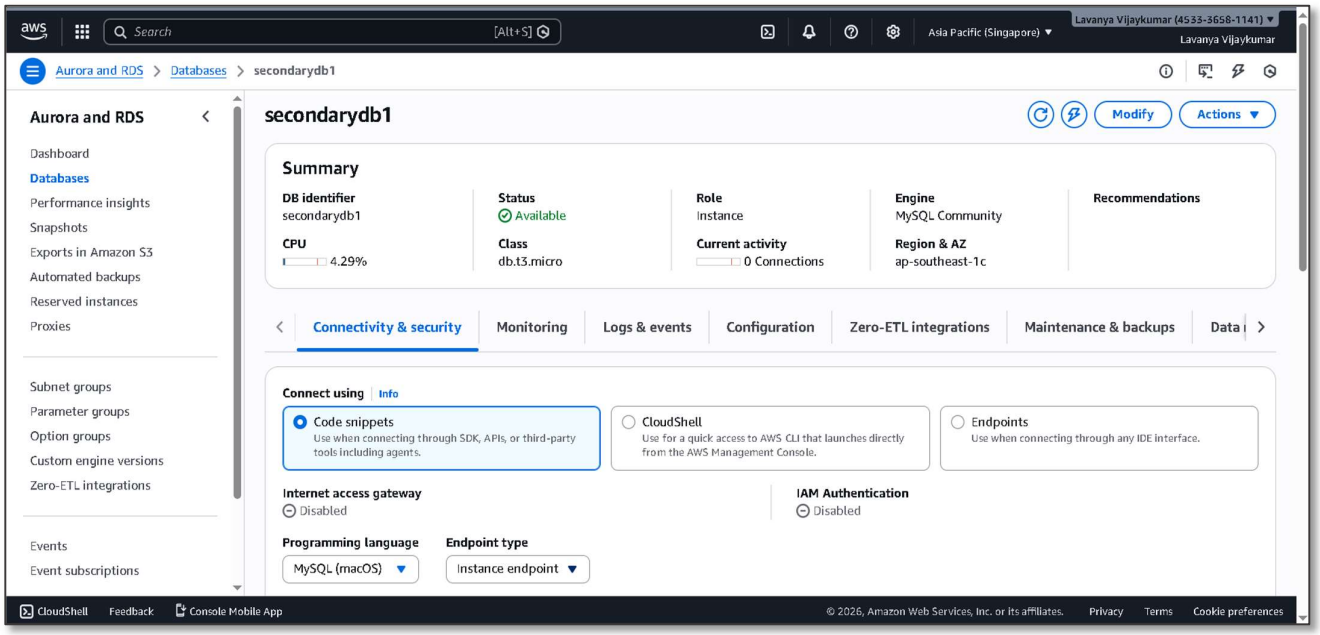


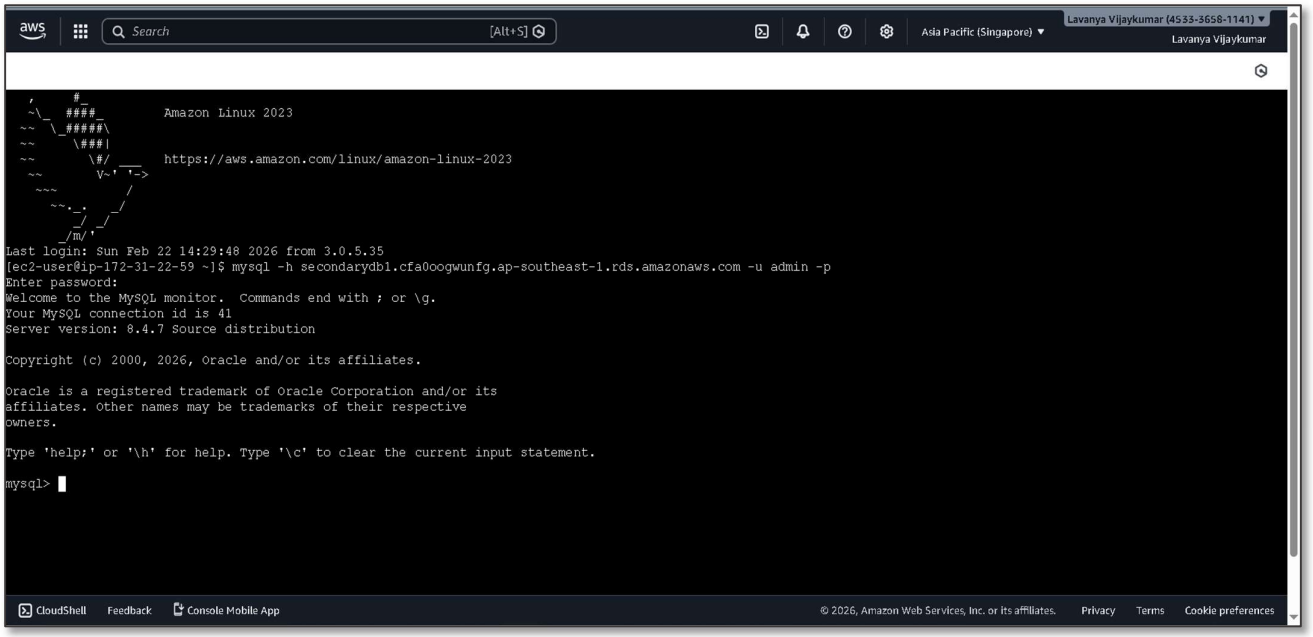
- Connect via SSH and install web server + MySQL client
- sudo dnf update -y
  sudo dnf install -y httpd
  sudo systemctl enable httpd
  sudo systemctl start httpd
  echo "Secondary Region EC2" | sudo tee /var/www/html/index.html
  sudo dnf install --nogpgcheck -y mysql-community-client



STEP 4:Secondary RDS Setup (Singapore)

- Take snapshot of Primary RDS → copy to Singapore → restore as SecondaryDB
- Security Group: allow Secondary EC2 IP on **3306**
- Test connection from Secondary EC2:
- mysql -h <SecondaryDB_Endpoint> -u admin -p

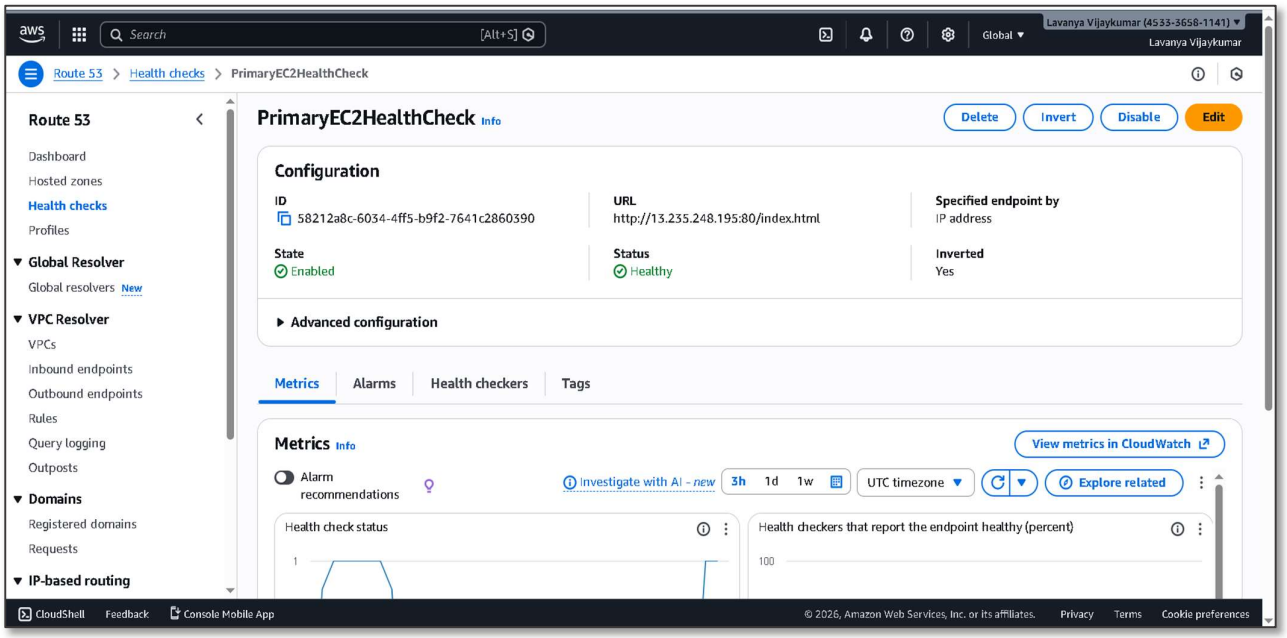




STEP 5:Route 53 Failover Setup

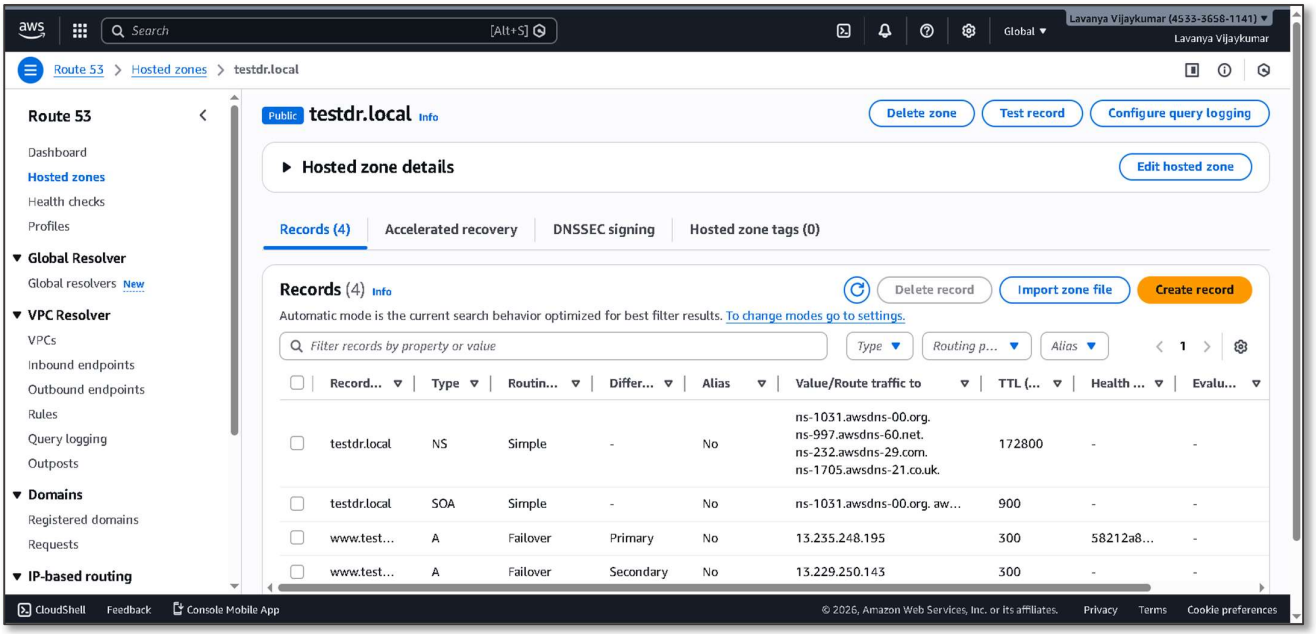
1.Create Health Check

- Go to **Route 53 → Health Checks → Create**
- Type: HTTP
- Endpoint: <Primary EC2 Public IP>/index.html
- Save



2.Create Hosted Zone & Records

- Domain: www.testdr.local (demo)
- **Primary Record:**
- Type: A → <Primary EC2 IP>
- Routing Policy: Failover → Primary
- Associate Health Check
- **Secondary Record:**
- Type: A → <Secondary EC2 IP>
- Routing Policy: Failover → Secondary



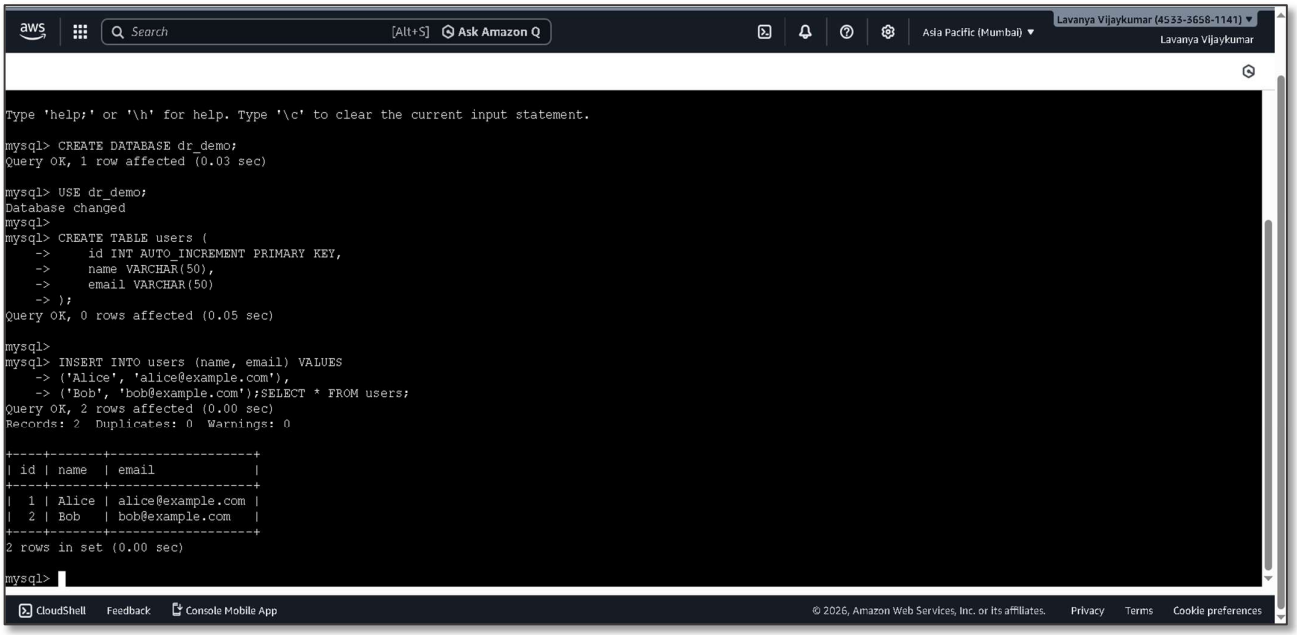
STEP 6:Insert Sample Data in Primary RDS

1.CREATE DATABASE dr_demo;
USE dr_demo;

```
CREATE TABLE users (
   id INT AUTO_INCREMENT PRIMARY KEY,
   name VARCHAR(50),
   email VARCHAR(50)
);

INSERT INTO users (name, email) VALUES
('Alice', 'alice@example.com'),
('Bob', 'bob@example.com');

SELECT * FROM users;
```

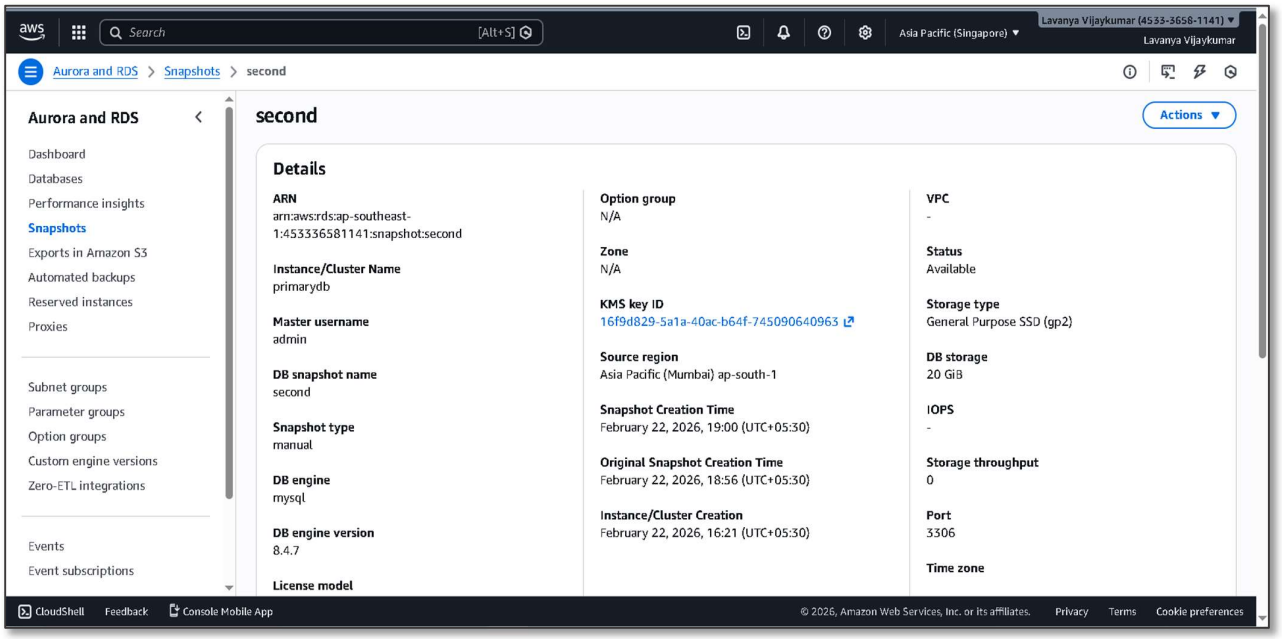


2. Take Snapshot of Primary DB and Restore Snapshot in Secondary Region

- RDS → PrimaryDB → Actions → Take Snapshot → PrimaryDB_snapshot

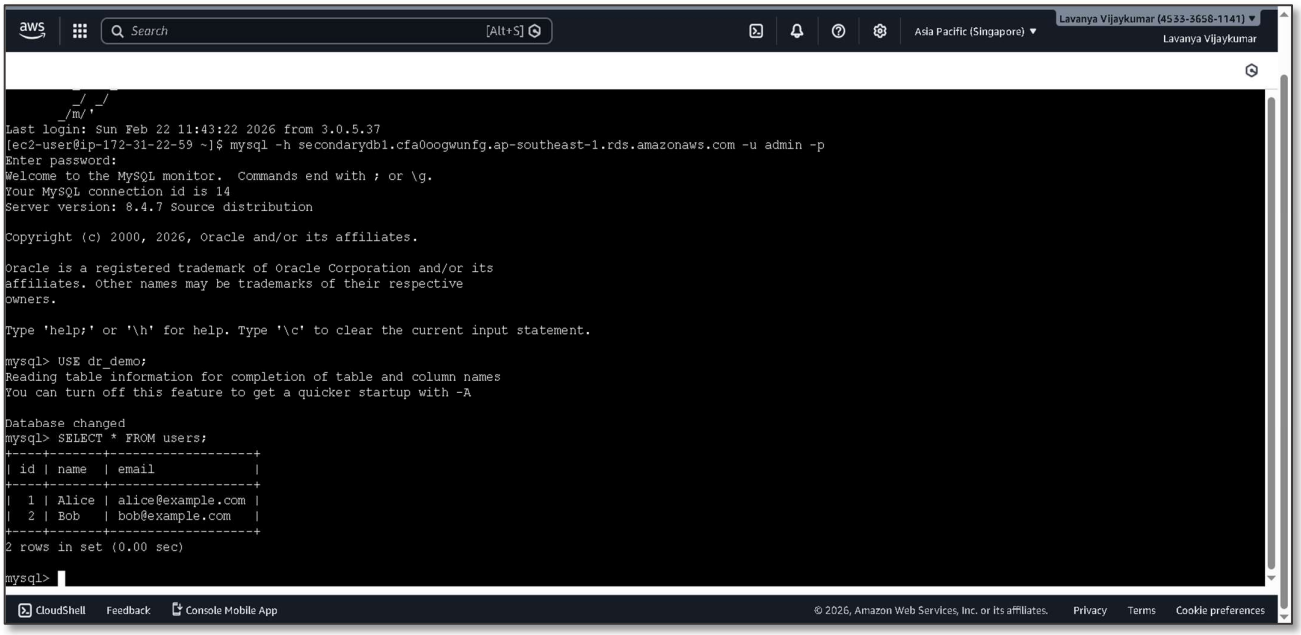
3.Restore Snapshot in Secondary Region

- Singapore → Snapshots → Restore → SecondaryDB
- Publicly Accessible: Yes, Security Group allows Secondary EC2 IP on 3306



4.Connect from Secondary EC2

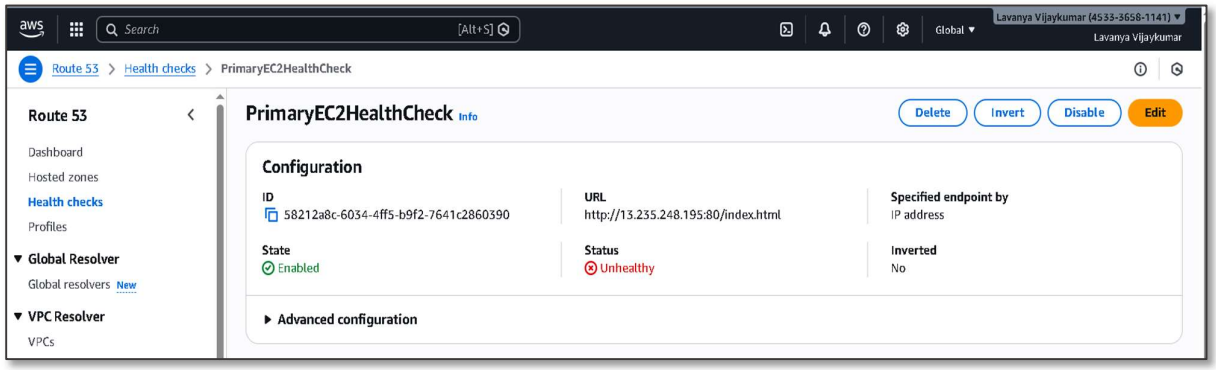
mysql -h <SecondaryDB_Endpoint> -u admin -p
USE dr_demo;
SELECT * FROM users;



STEP 7:Failover Testing (Web Traffic)

- Access Primary EC2 page: http://<Primary_IP> → shows Primary Region EC2

- Stop Primary EC2 → Health Check shows Unhealthy



- Access Secondary EC2 page: http://<Secondary_IP> → shows Secondary Region EC2

Troubleshooting:

Issue 1: MySQL client not found on EC2

Cause: EC2 AMI didn't include MySQL client by default

Solution: Install MySQL client using sudo dnf install --nogpgcheck -y mysql-community-client or Amazon Linux extras for MySQL

Issue 2: Unable to connect to Primary/Secondary RDS

Cause: Security group or RDS not publicly accessible

Solution: Enable Publicly Accessible for RDS and allow EC2 IP in RDS security group

Issue 3: GPG key errors while installing MySQL

Cause: Repository GPG key mismatch

Solution: Use --nogpgcheck during installation or download correct key

Conclusion:

The Multi-Region Disaster Recovery Plan project demonstrates how to design and implement a resilient cloud architecture on AWS that ensures high availability and business continuity. By deploying EC2 instances and RDS databases across two geographically separated regions, and configuring Route 53 health checks with failover routing, the project ensures that web traffic can be automatically redirected to a secondary region during primary failures.

The project also highlights manual database failover using RDS snapshots, providing hands-on experience in cross-region data replication and disaster recovery planning. This approach enhances understanding of high availability, failover strategies, and cross-region replication, while remaining cost-effective and suitable for academic or learning purposes.

In real-world production environments, the architecture can be extended with CloudFormation templates, Multi-AZ RDS, and real domain-based DNS to achieve fully automated failover for both web and database layers.